

Assignment 3

Due Date: December 10, 2010 by 5:30 pm

1. (12 points) You will be required to download and build the climate data objects (CDO) code. Please check the following page for details:

<http://www.mpimet.mpg.de/fileadmin/software/cdo/>

- a. Download and unpack cdo-1.4.3.tar.gz.
- b. Create the makefiles with the configure script. Make sure to set PREFIX so it doesn't get installed in a system-wide directory. Use the PGI compilers (GNU is the default so it will need to be changed).
- c. Build the package with make.
- d. Test the package.
- e. Install the package.
- f. Do a make clean. Are there any configuration files left (check for config.status and/or config.log)? If so, why?
- g. Clean out all of the object, binary and makefiles, as well as any configuration files (usually a variant of clean like distclean).
- h. Re-do steps a-f, but this time make sure it builds with the flag -fast. Do you get any errors in the build process anywhere? What type of error was it (e.g. compile, link, command-line, configure)?

For each step above, report:

- a. The command you typed at the command line (e.g. the entire configure line including all of the options)
 - b. Error messages, if any. If you run into any errors/complaints on the command line from the compiler, linker, configure script, command line, etc. report those, too. Don't bother with compiler or linker warnings, just report errors that caused the build to fail.
 - c. (keep) config.log, make.log, make_check.log (if there is one) and make_install.log
2. (22 points) In /globalscratch/dshaw/CMSC6940/assign3files/q2 on courtenay, there are two source files called shift_com.f and pdos.f. Copy these files back to your home directory on courtenay, compile these two files with pgf77 and report the errors that result when you use the default compiler flags (you can put them in a text file). What types of errors are they (i.e. compile-time, link or runtime)? Suggest how you think you could fix the errors.

For pdos.f, there is an analogous file called pdos_fixed.f that contains fixes for the errors that the compiler reported. Use the linux utility diff to compare the two files and fix the two errors. Run the code using ./a.out < pdos.inp and report any resulting errors. How many were reported? What type of error was it? Give a suggestion for fixing the error.

3. (15 points) It is a common thing that bugs get introduced to code through mixing of programming languages and copying and pasting lines or values. One such batch of errors has been introduced into the C package in the directories bug1, bug2, and bug 3 in /globalscratch/dshaw/CMSC6940/assign3files/q3. Copy these subdirectories back to your home directory on Courtenay and for each one, do the following:
- Use the makefile to build the source in each subdirectory.
 - Report the error that is given from the build process – what type of error is it (compile, link, any?)?
 - If there are no errors from the build process, run the code using the file `gen_xas_cpmd.inp`:
`./gen_xas < gen_xas_cpmd.inp`
 - Does the code run without errors or strange results (i.e. Inf, NaN)? If not, what type of error is reported?
 - If you get a core file, report the error from linux (usually some type of signal) and then use gdb to determine where in the code execution was stopped. Report the output from gdb by copying the session's output to a file.
 - BONUS: (3 points) Using the code from the lab (IntroMake, example 3) and the linux diff function, determine what the errors were and suggest a fix.
4. (10 points) In /globalscratch/dshaw/CMSC6940/assign3files/q4 on courtenay, there are 3 source files. Copy them to your home directory and create a Makefile to compile each source file into an executable (one executable for each source file). You can use whatever compilers and flags you choose as long as the codes build without compile or link errors on courtenay. Be sure to include a rule to clean up old executable and .o files.